CHAPTER 03

Back-of-the-envelope Estimation

Use powers of two, latency numbers and availability targets to size a system in minutes.

Jump to: What this chapter is about | Numbers you must know | Deep dive | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. What this chapter is about

Interviewers often say: "Estimate the QPS and storage for this system." You are not expected to be exact. You are expected to **pick sane assumptions, round aggressively, show your units, and reach an order of magnitude** that drives design choices (one DB or sharded? cache or not? how many servers?).

Why interviewers ask it: It proves you can turn a vague product into numbers and use those numbers to justify architecture.

2. Numbers you must know

- **Powers of two:** 2^10 = 1 Thousand (1 KB), 2^20 = 1 Million (1 MB), 2^30 = 1 Billion (1 GB), 2^40 = 1 Trillion (1 TB), 2^50 = 1 Quadrillion (1 PB).
- **Time:** 1 day = 86,400 s (about 10^5). 1 year about 3 x 10^7 s. Round a day to 100,000 s for quick math.
- **Latency ladder (rough, order of magnitude):** L1 cache 0.5 ns -> branch mispredict 5 ns -> L2 cache 7 ns -> mutex lock 100 ns -> main memory 100 ns -> compress 1 KB 10 us -> send 2 KB over 1 Gbps 20 us -> read 1 MB from memory 250 us -> same-DC round trip 500 us -> disk seek 10 ms -> read 1 MB from network 10 ms -> read 1 MB from disk 30 ms -> cross-continent packet round trip 150 ms.
- **Takeaways:** memory is fast, disk is slow; avoid disk seeks; simple compression is cheap; compress before sending over the network; data centers in different regions cost real time (about 150 ms).
- **Availability (SLA nines):** 99% = 14.4 min/day, 3.65 days/year; 99.9% = 1.44 min/day, 8.77 h/year; 99.99% = 8.64 s/day, 52.6 min/year; 99.999% = 0.86 s/day, 5.26 min/year.

3. Deep dive

Worked example: Twitter-like service

- **Assumptions:** 300 M monthly active users (MAU); 50% use it daily -> 150 M DAU; each user posts 2 tweets/day; 10% of tweets carry media; media kept 5 years.
- **Write QPS** = 150 M x 2 / 86,400 = about 3,500. **Peak QPS** = about 2 x average = 7,000.
- **Media storage/day** = 150 M x 2 x 10% x 1 MB = 30 TB/day. **5-year media** = 30 TB x 365 x 5 = about 55 PB.
- **Tweet text:** 300 bytes x 300 M tweets/day = 90 GB/day (tiny compared with media).
- **Conclusion:** text fits a sharded DB easily; media dominates -> object storage + CDN. Read QPS is far higher than write -> cache.

A repeatable recipe

- 1. Clarify scale: MAU/DAU, read:write ratio, payload size, retention, peak vs average.
- 2. Convert: users -> requests/day -> QPS (divide by about 10^5) -> multiply by 2-3 for peak.
- 3. Storage = objects/day x size x retention x replication factor (typically 3).
- 4. Bandwidth = QPS x payload size (ingress and egress separately).
- 5. Memory for cache: follow the 80/20 rule -> cache about 20% of daily read data.
- 6. Servers = peak QPS / QPS per server (e.g. 1-10 K for simple web handlers), then add headroom.
- 7. State the conclusion: what the numbers imply for the design.

Handy rules of thumb

- 1 M requests/day is about 12 QPS. 100 M/day is about 1,200 QPS. 1 B/day is about 12,000 QPS.
- A single commodity DB node handles roughly 1 K - 10 K simple QPS; a single Redis node about 100 K ops/s.
- 1 char = 1 byte (ASCII) or 2-4 bytes (Unicode). An int is 4 bytes, a long or timestamp 8 bytes, a UUID 16 bytes.
- A typical photo 200 KB - 2 MB; one minute of 720p video about 20-50 MB.
- Replication factor of 3 means raw storage is 3x logical storage.

4. Common mistakes

- Hiding behind precision: 86,400 vs 100,000 does not matter; write a clean assumption and move on.
- Forgetting units (bits vs bytes, KB vs MB) and forgetting peak vs average.
- Doing the math but never saying what it means for the architecture.
- Ignoring replication, metadata and index overhead in storage estimates.

5. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	3 min	"Before estimating, I'll confirm the scale inputs: monthly and daily active users, read-to-write ratio, object size, retention, and whether I should size for average or peak. I'll write my assumptions on the board so we can adjust any of them."
2. High-level design	5 min	"Convert users into requests per day, then divide by about 10^5 seconds to get QPS and multiply by 2 or 3 for peak. Storage is items per day times size times retention times 3 for replication. Bandwidth is QPS times payload."
3. Deep dive	8 min	"Now I use the numbers to decide architecture. For example, about 3,500 write QPS fits a sharded database comfortably, but 55 PB of media means object storage and a CDN. A read QPS 10x higher than writes tells me caching matters."
4. Wrap up	2 min	"So the design needs a cache tier, sharded storage for text, object storage plus CDN for media, and enough servers for peak with headroom. I'm happy to adjust if you give me different assumptions."

6. 60-second recap / practice checklist

- Round numbers, write assumptions, label units, state peak = 2-3x average.
- QPS = daily requests / 10^5. Storage = items x size x retention x 3.
- Memorise the latency ladder and the nines table; finish every estimate with 'so the design needs ...'.
- Practice: estimate QPS and storage for Instagram, WhatsApp, YouTube, a URL shortener, in under three minutes each.

7. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Latency numbers every programmer should know (interactive)
• Jeff Dean latency numbers (gist)
• GeeksforGeeks: Capacity estimation